RBE 4701: AI for Robotics A26 Project 1 Report

Team 04: Patrick Beaudin, Marcus Gafka, Elliot Scher

1. Structure of the approach:

Our goal was to develop a single approach capable of solving all five variants rather than implementing a separate solution for each scenario. We initially explored several approaches, including breadth-first search (BFS), A*, minimax, expectimax, MDP value iteration, and policy iteration.

We implemented our character as a custom character subclass of CharacterEntity, allowing the decision-making logic to be contained in one player implementation. The final architecture consists of three primary components: path planning, state evaluation, and monster avoidance. A* determines efficient paths toward the exit, while value iteration is used when monsters that are close by make the shortest path unsafe. The character dynamically switches between these behaviors based on the current board configuration.

Through testing, we found that the required success rates could be achieved without using bombs to eliminate monsters or modify the environment. Since the walls and exit remain fixed throughout a game, the problem can be treated as a fixed set of relevant board configurations. This allowed us to precompute states rather than repeatedly solving the entire decision problem during runtime.

2.1 Precomputation and Value Iteration

Because the board does not change, we model the problem as a Markov Decision Process (MDP) in which the state is represented by the character and monster locations. The character has nine possible actions: movement to any of the eight neighboring cells or remaining stationary. Character movement is deterministic, but uncertainty primarily comes from the monster's movement.

The reward structure strongly prioritizes survival and reaching the exit:

- Reaching the exit: reward = +10,000
- Entering the monster's attack range: reward = −10,000
- Every other step: reward = −1

The death state is based on the monster's attack range rather than requiring both entities to occupy the same cell.

Value iteration runs once, during the first character action when monsters are present. It performs up to 100 iterations and terminates early when the maximum change in any cell's value is below 1.0. The resulting value table is cached and reused throughout the game.

To reduce computation, only states where the character and monster are within a Chebyshev distance of seven (7) are explicitly calculated. The map contains 136 open cells, giving (136^2=18,496) possible character/monster pairs. The seven-tile restriction reduces this to 11,488 states, approximately 38% fewer.

The same radius is used during gameplay. If no monster is within seven tiles, the character follows the shortest A* path to the exit. If a monster is nearby, the character evaluates its nine possible actions using the precomputed value table. The radius of seven was selected experimentally: larger values caused unnecessarily conservative behavior, while smaller values caused the character to react too late.

2.2 Choosing Distance Metrics

Because both the character and monsters can move in eight directions, diagonal movement has the same cost as horizontal or vertical movement. Chebyshev distance is therefore used for proximity-based decisions, including the monster relevance radius, attack range, chase range, and distance to the nearest wall.

Chebyshev distance alone is insufficient for path planning because walls can force a longer route. We therefore use BFS to compute shortest-path distances through the map using 8-connected movement. A BFS starting at the exit calculates the shortest distance from every open cell to the exit, allowing these values to be reused throughout gameplay. BFS is also used locally when predicting monster movement, since geometric distance does not necessarily correspond to traversable distance.

For A*, Chebyshev distance is used as the heuristic because it does not overestimate remaining path cost when diagonal and straight moves have equal cost. Euclidean distance is used only as a final tie-breaker when multiple actions have equivalent Chebyshev distances.

2.3 Predicting Monster Movement

The value of an action depends on both the character's resulting position and the monster's predicted response. We therefore use a common monster prediction model for all variants. When a monster is within a Chebyshev distance of two from the character, it is assumed to chase the character. The predicted move is the neighboring cell that most reduces its shortest-path distance to the character. A local BFS with a radius of three is used to calculate this distance.

When the monster is farther than two tiles away, it is modeled as having an equal probability of moving to any open neighboring cell or remaining stationary. This creates the stochastic transition model required by the MDP.

The chase threshold of two was selected based on the behavior of the most dangerous monster in Variant 4. Applying this threshold to every monster makes the prediction intentionally conservative. When multiple chase moves have equal shortest-path distance, the algorithm predicts the move that places the monster farthest from a wall. This logic was determined experimentally to reduce the frequency that the character becomes trapped in corners. Remaining ties are distributed evenly among the possible moves.

2.4 Rewards, Discount Factor, and Thresholds

The reward values ensure that survival and reaching the exit dominate the small cost of individual movements. The discount factor is gamma=0.9 so future rewards have progressively less influence on the current decision.

Several thresholds are used throughout the algorithm:

- Death zone = 1: distance at which the monster can attack
- Chase threshold = 2: distance at which the monster is modeled as chasing
- Local search radius = 3: BFS region used to determine chase movement
- Relevance radius = 7: distance at which monster-aware planning becomes active

The seven-tile relevance radius was determined experimentally, while the other thresholds correspond more directly to the movement and detection behavior of the game. When two actions have the same MDP value, a small Euclidean-distance-based penalty encourages progress toward the exit without overriding meaningful differences in state value.

3. Experimental evaluation:

Testing the algorithm was time consuming because each trial normally required stepping through the Pygame interface until the end of the game. Rendering also significantly increased execution time. To address this, we developed a headless testing script that could run thousands of trials without rendering the game. This allowed us to evaluate changes much more efficiently.

We also used GitHub for version control and integrated the testing script with pytest. Tests were run through our CI/CD workflow, with pull requests requiring the tests to pass before changes could be merged. This helped prevent changes to the algorithm from introducing bugs into the main code.

Results of 1000 tests of each variant:

Totals:

Variant 1: 1000/1000 escaped, success rate 100.0%
Variant 2: 997/1000 escaped, success rate 99.7%
Variant 3: 938/1000 escaped, success rate 93.8%
Variant 4: 664/1000 escaped, success rate 66.4%
Variant 5: 817/1000 escaped, success rate 81.7%

Across all 5,000 trials, the algorithm achieved an overall success rate of 86.3%, with the highest failure rates occurring in Variants 4 and 5. The results show that the same algorithm can successfully handle substantially different game configurations without requiring variant-specific decision-making logic. The lower success rates in Variants 4 and 5 indicate that the more complex monster behaviors remain the most challenging cases, but the shared planning and prediction model was still able to achieve meaningful success without using bombs to kill monsters or destroy walls to change the map.

RBE 4701: AI for Robotics A26 Project 2 Report

Team 04: Patrick Beaudin, Marcus Gafka, Elliot Scher

1. Structure of the approach:

Our goal was again to develop a single approach capable of solving all five variants rather than implementing a separate solution for each scenario. Unlike Project 1, every Project 2 variant requires the character to destroy walls to reach the exit. The agent therefore has to learn when and where to place bombs while also surviving its own explosions and avoiding monsters.

We implemented our character as a custom subclass of `CharacterEntity` using approximate Q-learning with a linear Q-function. Every step, the character evaluates ten possible actions: the eight movement directions, remaining stationary, and placing a bomb. The same features and decision-making code are used for all variants, while each variant uses its own learned weights.

A finite state machine selects the weight vector used by the Q-function. The four states are checked in priority order:

- **ESCAPE_BLAST:** the character is inside the cross of a ticking bomb
- **WAIT_BLAST:** a bomb or explosion exists elsewhere on the map
- **BREACH:** the next cell on the route to the exit is a solid wall
- **NAVIGATE:** all other situations, and always on the first step

Each state has its own weight vector. The state machine does not restrict the available actions, it only determines which weights score them. Each state also has a no-monster version, such as `NAVIGATE_NOMON`, resulting in eight weight vectors total. Monster behavior is instead represented through features in every state so that monster danger can be considered alongside progress toward the exit.

A safety shield is applied before action selection to remove illegal actions and actions that would result in certain or highly likely death from an explosion. The complete decision process is therefore: determine the state, compute features, mask unsafe actions, update the previous Q-value during training, and select the highest-scoring remaining action.

2.1 Approximate Q-Learning Formulation

The Q-function is linear in the features, with one weight vector for each state machine state. After each step, the weights associated with the previous state are updated using approximate Q-learning:

The error is clipped to ±50. We used:

- gamma=0.9
- learning rate α=0.01

The reward structure provides feedback before reaching the exit:

- Every step: −1
- Progress toward the exit: +1
- Each wall destroyed: +5
- Reaching the exit: +100
- Dying: −100

The progress reward was necessary because the exit is located behind several rows of walls. With only terminal rewards, a character starting from zero weights rarely reaches the exit through random exploration and receives little information about which actions lead toward a solution.

2.2 Features and Breach Distance

The primary quantity used by the feature set is **breach distance**, defined as the cost of the shortest path from a cell to the exit when solid walls have a cost of 6 instead of 1. Walls inside a ticking bomb's blast cross are treated as open. This allows the agent to compare walking and bombing using the same distance metric.

The features describe the transition (s,a) rather than only the destination cell. Features are evaluated at the destination, in the predicted state immediately after the action, or when the relevant bomb detonates. The initial feature set contained 29 features in six groups:

- **Base:** breach distance before and after the action, distance at the exit, distance from the map center, and fraction of walls remaining
- **Action:** whether the action is a bomb or stay
- **Space:** reachable cells, safe neighboring cells, and dead ends
- **Bomb:** breach distance gained, walls hit, wasted bombs, opened region, and distance to useful and escapable bombing locations
- **Survival:** blast-cross occupancy, time to safety, moves remaining before detonation, safe cells reachable in time, and escape feasibility

- **Monster:** BFS distance to the nearest predicted monster position, nearby monsters, whether the nearest monster is closing in, and predicted collision

All features are normalized to approximately [0,1] or [-1,1]. Monster distances use BFS through open cells rather than geometric distance, since walls can separate cells that are geometrically close.

2.3 Safety Shield

Bomb deaths are difficult to learn because the action causing the death can occur several steps before the resulting -100 reward. We therefore added a safety shield that masks actions before comparing their Q-values.

The shield removes:

- **Illegal actions:** moving off the map or into a wall, or placing a bomb while another bomb is active
- **Level 1:** actions resulting in certain death by explosion during the current or next step
- **Level 2:** actions that leave insufficient time to escape before detonation

If all legal actions are masked, the shield is relaxed and the highest-Q action is selected. Level 2 shielding is active during both training and final evaluation. The shield only removes actions that are known to be fatal, it does not select the character's action. This allows the Q-function to focus learning on monster avoidance and useful bombing locations rather than repeatedly learning basic blast survival.

2.4 Training

We trained each variant from all-zero weights. Training was performed headless, allowing tens of thousands of episodes to be run without rendering. Exploration used epsilon-greedy action selection, with epsilon decaying linearly from 0.3 to 0.02 over the first 80% of training. Episodes were limited to 500 steps.

We ran four independent training runs per variant:

- **a, b:** a=0.01, epsilon from 0.3 to 0.02, different random seeds
- **explore:** epsilon from 0.5 to 0.05
- **slow:** a=0.003

Every 10 minutes, the current weights were evaluated over 10 greedy games with learning disabled. The best checkpoint was retained because performance did not always improve. In Variants 4 and 5, later training sometimes caused a previously successful policy to degrade. Each candidate checkpoint was then tested on the same 100 games.

2.5 Feature Pruning

The initial 29-feature model with 8 state weight vectors contains 232 weights per file. Several features had little effect on action selection, while others were highly correlated. We therefore pruned features from the trained weights for each variant.

The pruning process was:

1. **Influence:** count how often setting each feature's weight to zero changes the selected action.
2. **Impact:** remove each feature individually and measure escapes lost over 100 greedy games..
3. **Pruning:** remove features that were the least impactful while maintaining the escape-rate tolerance and reducing average steps to escape.
4. **Validation:** compare the full and pruned feature sets on a set of 1000 games.

   The weights were not retrained during pruning. The final models use:

   - 4 features for Variant 1
   - 26 features for Variant 2
   - 22 features for Variant 3
   - 17 features for Variant 4
   - 23 features for Variant 5

Variant 1 requires only a bias, breach distance to the exit, breach distance gained by a bomb, and escape margin. Pruning reduced its average escape time from 79 to 71 steps and reduced Variant 2 from 101 to 77 steps, although Variant 2's validation success rate decreased from 96% to 93.5%. Variants 3 through 5 maintained similar success rates with fewer features.

3. Experimental evaluation:

As in Project 1, we evaluated the agent using a headless testing script. During testing, learning is disabled and the character always selects the highest-Q action. Each game uses a different monster random seed allowing the results to be reproduced.

We ran the final weights for six hours per variant, with all five variants running in parallel. The resulting number of games varies because the variants take different amounts of time to complete.

Variant	Games	Escaped	Died	Timed out	Success rate	Avg. steps
1	291,072	291,072	0	0	100.0%	71.0
2	119,181	111,234	7,947	0	93.3%	77.6
3	100,103	97,656	2,403	44	97.6%	97.1
4	68,452	46,184	22,268	0	67.5%	149.9
5	38,103	34,448	3,534	121	90.4%	191.3

All five variants exceeded the 50% success threshold, with an average success rate of 89.8%. As in Project 1, Variant 4 was the most difficult because its aggressive monster begins chasing the character within two cells while the character must also destroy several walls. Its 67.5% success rate was similar to the Project 1 result of 66.4%, despite the additional requirement of clearing walls. Variant 5 improved from 81.7% in Project 1 to 90.4%.

The six-hour evaluation also produced results close to the smaller validation tests. The final escape rates were within approximately 3% of the 200-game validation results for every variant except Variant 4, where the validation rate was 62.5% compared with 67.5% in the six-hour evaluation.